

USF: Unit-Streamed Fine-Tuning of Large Language Models with Exact Gradients on Consumer GPUs

Nicolas Andrey Orozco Giraldo, Luis Eduardo Muñoz Guerrero

Universidad Tecnológica de Pereira
Pereira, Colombia

nicolas.orozco@utp.edu.co, lemuno zg@utp.edu.co

ORCID: 0000-0002-6332-9757, 0000-0002-9414-6187

Yony Fernando Ceballos

Facultad de Ingeniería, Grupo de Ingeniería y Sociedad
Universidad de Antioquia
Medellín, Colombia

yony.cebillos@udea.edu.co

ORCID: 0000-0001-5787-8832

Resumen—El *fine-tuning* de un modelo de lenguaje de 7 B parámetros requiere convencionalmente entre 8 y 16 GB de memoria de GPU, dado que la base congelada debe permanecer residente durante los pases hacia adelante y hacia atrás. Ese requisito es precisamente lo que excluye a quien no tiene hardware de nivel centro de datos de ajustar modelos de esta escala en absoluto, sin importar cuán pocos parámetros pretenda entrenar. Demostramos que el requisito es un artefacto de la implementación y no una necesidad matemática. La diferenciación en modo inverso compone productos vector-jacobiano *locales*: el gradiente en la capa i depende únicamente de la entrada de esa capa, de sus pesos y del cotangente entrante, nunca de las restantes $L - 1$ capas. Presentamos *Unit-Streamed Fine-Tuning* (USF; ajuste fino por streaming de unidades), que mantiene exactamente una unidad de pesos residente a la vez, transmitida desde disco, y demostramos que los gradientes resultantes son *exactos* (Teorema 3) y no aproximados. La consecuencia es una cota de memoria independiente de la profundidad del modelo y de su número total de parámetros: solo importa la unidad individual más grande (Teorema 7). Demostramos además que invertir el orden de los bucles (*layer-major*, Teorema 13) divide el tráfico de disco por el factor de acumulación de gradiente G dejando los gradientes idénticos bit a bit, lo que traslada el régimen de estar limitado por E/S a estarlo por cómputo. En una GTX 1050 móvil (3,94 GB, sin tensor cores) —una elección deliberadamente desfavorable, dado que una cota que se cumple aquí se cumple en prácticamente cualquier GPU que un practicante pueda poseer— ajustamos Qwen2.5-7B —15,2 GB de pesos en disco— con un pico de 2,2 GB en el asignador, o 2,9 GB de ocupación total del dispositivo incluyendo el contexto CUDA, reduciendo la perplejidad en datos reservados de 9,52 a 4,46; la exactitud del esquema layer-major se verifica sobre el propio modelo de 7 B ($\max |\Delta| = 0$ en 1400 tensores) y produce una aceleración medida de $1,40\times$. Un modelo de coste predictivo concuerda con las mediciones dentro de un 6 %. USF es agnóstico al adaptador: un adaptador espectral y LoRA se ejecutan sin modificación alguna. Bajo el mismo presupuesto, la cuantización a 4 bits no cabe (requiere 5,43 GB). La cota predice que Llama-3-

70B cabe en la misma tarjeta con la granularidad implementada.

Index Terms—*Parameter-efficient fine-tuning*, entrenamiento con memoria limitada, modelos de lenguaje de gran tamaño, gradientes exactos, cómputo *out-of-core*, diferenciación automática, hardware de consumo.

I. INTRODUCCIÓN

EL *parameter-efficient fine-tuning* (PEFT; ajuste fino eficiente en parámetros) redujo en órdenes de magnitud la huella *entrenable* de la adaptación de modelos de lenguaje: LoRA [1] actualiza muy por debajo del 1 % de los pesos, y los adaptadores [2] o el *prompt tuning* [3] actualizan aún menos. La huella *residente*, sin embargo, no siguió el mismo camino. Sea cual sea el adaptador, la base congelada debe estar presente durante los pases hacia adelante y hacia atrás, de modo que adaptar un modelo de 7 B en precisión media sigue exigiendo entre 8 y 16 GB de memoria de dispositivo. El efecto práctico es un censo de hardware: quien dispone de una GPU de 4 GB queda excluido de trabajar con modelos de 7 B, con independencia de cuán pocos parámetros pretenda entrenar.

La cuantización ataca la constante [4], [5], comprimiendo la base residente en un factor de cuatro a ocho. No ataca el acoplamiento. Un modelo de 7 B en 4 bits sigue ocupando varios gigabytes, uno de 70 B en 4 bits sigue excediendo cualquier tarjeta de consumo, y la compresión se paga en precisión. Los marcos de *offloading* como ZeRO-Infinity [6] sí trasladan los parámetros fuera del dispositivo, pero fueron diseñados para un régimen distinto —entrenar *todos* los pesos en un clúster— donde el problema dominante es el particionado del estado del optimizador.

I-A. La observación

Este artículo parte de una propiedad elemental de la diferenciación en modo inverso [7], [8] que, hasta donde sabemos, no se ha explotado hasta sus últimas consecuencias en este contexto. La retropropagación es una composición de productos vector-jacobiano (VJP, del inglés *vector-Jacobian product*) locales. Para derivar a través de la capa i se requieren la entrada de esa capa, sus pesos y el cotangente entrante, y nada más. Las restantes $L - 1$ capas son irrelevantes *en ese instante*. Mantener el modelo completo residente es, por tanto, una decisión tomada por velocidad, no un requisito impuesto por la corrección.

Tomamos esa observación al pie de la letra. USF mantiene exactamente una *unidad* de pesos en memoria del dispositivo a la vez, la transmite desde disco, computa, la libera y la vuelve a transmitir durante el pase hacia atrás. La pregunta inmediata es si esto cuesta precisión. No la cuesta: demostramos (Teorema 3) y verificamos ($\max |\Delta| = 0$) que los gradientes son idénticos a los de la diferenciación automática ordinaria sobre el grafo completo. USF no es una aproximación.

I-B. La consecuencia

Una vez desacoplada la residencia de la corrección, la cota de memoria cambia de forma. El pico lo fija la unidad individual más grande:

$$\text{VRAM}_{\text{pico}} \approx \max_u |W_u| \cdot s + |\theta| \cdot s_{\text{opt}} + A, \quad (1)$$

que no depende ni de la profundidad L ni del tamaño total $\sum_i |W_i|$ (Teorema 7 y sus corolarios). Añadir capas a un modelo no incrementa su requisito de memoria; incrementa únicamente el tiempo y el tráfico de disco. El tamaño del modelo deja de ser un problema de memoria y pasa a ser un problema de almacenamiento y tiempo. Consideramos que este desacoplamiento, y no ningún componente de ingeniería concreto, constituye la aportación de este trabajo.

Un segundo resultado se sigue de la misma localidad. Puesto que la acumulación de gradiente es una suma, y las sumas son invariantes al orden, los dos bucles anidados de un paso de entrenamiento —sobre micro-lotes y sobre capas— pueden intercambiarse. Mantener una unidad residente mientras los G micro-lotes pasan por ella divide el tráfico de disco por G con gradientes idénticos bit a bit (Teorema 13). Este reordenamiento carece de sentido cuando el modelo cabe en memoria, lo que presumiblemente explica que la literatura no lo contemple; bajo streaming es la optimización dominante, y traslada el sistema de estar limitado por E/S a estarlo por cómputo.

I-C. Aportaciones

- 1) **Un resultado de independencia del tamaño.** Demostremos que la memoria necesaria para ajustar un modelo congelado es función únicamente de su unidad transmisible más grande, independiente de la profundidad y del número total de parámetros (Teorema 7, Corolario 8, Corolario 9).
- 2) **Exactitud, demostrada y verificada.** USF preserva los gradientes exactamente (Teorema 3); medimos $\max |\Delta| = 0$, incluso sobre el propio modelo de 7 B (Sección XI).

- 3) **Streaming layer-major.** Intercambiar el orden de los bucles divide la E/S por el factor de acumulación G sin coste alguno en corrección (Teorema 13), lo que cambia el régimen de rendimiento (Sección IX).
- 4) **Un modelo de coste predictivo** que concuerda con la medición dentro de un 6 % (Sección IX), lo que permite presentar nuestras afirmaciones de escalado como predicciones y no como extrapolaciones.
- 5) **Un sistema agnóstico al adaptador y al modelo.** La misma infraestructura ejecuta LoRA y un adaptador espectral sin modificación (Corolario 5), y selecciona automáticamente su granularidad de streaming a partir de la memoria disponible (Sección X).

Demostremos 7 B en una tarjeta de 4 GB; *predecimos*, a partir de una cota demostrada y un modelo de coste validado, que 70 B cabe en la misma tarjeta con la maquinaria que implementamos; e identificamos con precisión qué extensión no implementada requeriría un modelo de 405 B (Sección XII). No presentamos estas dos últimas como resultados.

II. TRABAJO RELACIONADO

Parameter-efficient fine-tuning. LoRA [1] y sus descendientes [9], los adaptadores [2] y el *prompt tuning* [3] minimizan los parámetros *entrenables*. Son ortogonales a este trabajo: USF gobierna dónde reside el cómputo, no qué se adapta, y el Corolario 5 lo formaliza. Evaluamos LoRA y un adaptador espectral bajo USF sin modificar ninguno de los dos.

Cuantización. QLoRA [4] y LLM.int8() [5] reducen la base residente comprimiéndola. El eje es distinto al nuestro en dos aspectos. Primero, la cuantización compra memoria con *precisión*, mientras que USF la compra con *tiempo* y es exacto (Corolario 4). Segundo, mejora la constante pero preserva el acoplamiento con el tamaño del modelo; la Sección XI-F recoge la consecuencia concreta: en nuestro presupuesto, 4 bits resulta inviable y USF no. Ambas técnicas son componibles y no las presentamos como alternativas.

Offloading y particionado. ZeRO [10], ZeRO-Offload [11] y ZeRO-Infinity [6] establecieron que los parámetros y el estado del optimizador pueden residir fuera del dispositivo, y son el precedente más cercano. El régimen difiere en el punto que importa. Esos sistemas se orientan a entrenamientos en los que todos los pesos son entrenables, distribuidos sobre un clúster; su dificultad central, y su aportación principal, es particionar el estado del optimizador, cuyo tamaño es proporcional al número de parámetros entrenables. En nuestro régimen $|\theta| \ll |W|$, de modo que no hay estado del optimizador que merezca particionarse, y el problema se reduce por completo a la residencia de W . Lo que aportamos no es, por tanto, el *offloading* en sí, sino (i) la cota formal y su independencia del tamaño del modelo (Teorema 7), (ii) la jerarquía de granularidad que determina la viabilidad (Teorema 10), y (iii) el reordenamiento layer-major (Teorema 13), que carece de propósito en un régimen donde el modelo es residente. Las bibliotecas de despacho de producción como *accelerate* [12] ofrecen ubicación en CPU/disco a nivel de módulo, orientada principalmente a inferencia; la Sección XI-F recoge nuestras mediciones con esa vía.

Recomputación. El checkpointing de activaciones [13] y los esquemas clásicos REVOLVE [14] intercambian cómputo por memoria descartando y recomputando *activaciones*. Esto es ortogonal y complementario: nosotros empleamos checkpointing y, además, descartamos y retransmitimos los *pesos*. Hasta donde sabemos, el compromiso almacenamiento/recomputación no se había aplicado antes a pesos congelados acompañado de una garantía de exactitud.

Diseño consciente de la E/S. FlashAttention [15] demostró que reorganizar un cómputo en torno a su tráfico de memoria —sin alterar su resultado— produce mejoras sustanciales, y que la exactitud merece preservarse en lugar de sacrificarse. USF comparte esa filosofía en otro nivel de la jerarquía: FlashAttention minimiza el tráfico entre la memoria en chip y la de alto ancho de banda dentro de la atención, mientras que USF gobierna el tráfico entre disco y dispositivo para los pesos del modelo completo. El Teorema 13 es un resultado de planificación consciente de la E/S en ese mismo espíritu, y la Sección IX sigue el razonamiento roofline de [16] para identificar qué recurso limita.

Pipelining. GPipe [17] y enfoques relacionados [18] reparten capas entre dispositivos y solapan micro-lotes para llenar el *pipeline*. Superficialmente, layer-major (Teorema 13) recuerda a una planificación de *pipeline*; el propósito está invertido. El *pipelining* distribuye un modelo demasiado grande para un dispositivo entre varios; layer-major conserva un único dispositivo y amortiza el coste de *releer* el modelo, un gasto que solo existe cuando los pesos se transmiten.

III. PRELIMINARES

Sea un transformer congelado de L capas que computa $h_0 = E(x)$, $h_i = f_i(h_{i-1}; W_i, \theta_i)$ para $i = 1, \dots, L$, y $\ell = C(h_L, y)$, donde W_i son los pesos *congelados* de la capa i y θ_i los parámetros *entrenables* del adaptador. Puesto que la base está congelada, $\nabla_{W_i} \ell$ nunca se requiere: solo $\nabla_{\theta} \ell$. Asumimos el régimen PEFT $|\theta| \ll |W|$ en todo el trabajo, y escribimos s para los bytes por escalar ($s = 2$ en fp16), B para el tamaño de micro-lote, S para la longitud de secuencia, d para el ancho del modelo y G para el número de pasos de acumulación de gradiente.

Definición 1 (Unidad transmisible). *Una unidad u es un subconjunto de pesos que puede cargarse y liberarse de forma independiente, y $\mathcal{U}$ denota una partición de $\{W_i\}$ en tales unidades. Consideramos tres granularidades anidadas: capa (un bloque decodificador completo), sub-capas (el bloque de atención y el bloque MLP por separado) y matriz (una única proyección).*

Definición 2 (Operadores de streaming). *CARGAR(u) materializa W_u desde disco a la memoria del dispositivo con un coste de E/S de $|W_u|$ bytes; LIBERAR(u) devuelve W_u a un dispositivo marcador, liberando la memoria sin coste de E/S. Ambos están estrictamente anidados: no se carga ninguna otra unidad entre un CARGAR(u) y su LIBERAR(u) correspondiente.*

El Algoritmo 1 enuncia el algoritmo base. El pase hacia adelante no construye grafo de diferenciación: se descartan

Algoritmo 1 USF (batch-major, granularidad de capa)

```

1:  $h \leftarrow E(x)$ 
2: para  $i = 1$  hasta  $L$  hacer ▷ adelante, sin grafo
3:    $c_i \leftarrow h$  ▷ checkpoint: entrada de la capa  $i$ 
4:   CARGAR( $W_i$ );  $h \leftarrow f_i(h; W_i, \theta_i)$ ; LIBERAR( $W_i$ )
5: fin para
6:  $\ell \leftarrow C(h, y)$ ;  $g \leftarrow \partial \ell / \partial h_L$ 
7: para  $i = L$  hasta 1 hacer ▷ atrás, retransmitiendo
8:    $x \leftarrow c_i$  con seguimiento de gradiente activado
9:   CARGAR( $W_i$ )
10:   $\hat{h} \leftarrow f_i(x; W_i, \theta_i)$  ▷ recomputar con grafo
11:   $\hat{h}$ .RETROPROPAGAR( $g$ ) ▷ acumula  $\partial \ell / \partial \theta_i$ 
12:  LIBERAR( $W_i$ );  $g \leftarrow x$ .grad
13: fin para

```

las activaciones y los pesos, y cada capa se recomputa —con sus pesos retransmitidos— durante el pase hacia atrás.

IV. EXACTITUD

La propiedad que define a USF es que el streaming no es una aproximación. Esto lo separa categóricamente de la cuantización, que intercambia memoria por precisión.

Teorema 3 (Exactitud del gradiente). *Sea $\nabla_{\theta} \ell^{\text{full}}$ el gradiente producido por diferenciación en modo inverso sobre el grafo de cómputo completo con todos los W_i residentes, y sea $\nabla_{\theta} \ell^{\text{USF}}$ el gradiente producido por el Algoritmo 1. Entonces, en aritmética exacta,*

$$\nabla_{\theta} \ell^{\text{USF}} = \nabla_{\theta} \ell^{\text{full}}. \quad (2)$$

Demostración: La diferenciación en modo inverso computa, para cada capa i , el par

$$\left(\frac{\partial \ell}{\partial \theta_i}, \frac{\partial \ell}{\partial h_{i-1}} \right) = \text{VJP}_{f_i} \left(h_{i-1}, W_i, \theta_i; \frac{\partial \ell}{\partial h_i} \right). \quad (3)$$

La aplicación en (3) es *local*: es función de (a) el punto de evaluación h_{i-1} , (b) los parámetros W_i, θ_i , y (c) el cotangente entrante $\partial \ell / \partial h_i$, exclusivamente. En particular, no depende de W_j para ningún $j \neq i$.

El Algoritmo 1 suministra las tres sin alteración. Para (a), el checkpoint $c_i = h_{i-1}$ se registra durante el pase hacia adelante, de modo que la recomputación se evalúa en el punto idéntico, sin aproximación. Para (b), CARGAR(W_i) restaura W_i bit a bit, puesto que relea el mismo tensor del disco, mientras que θ_i es residente y no se modifica dentro de un paso. Para (c), $g = \partial \ell / \partial h_i$ se cumple por inducción hacia atrás desde $g_L = \partial \ell / \partial h_L$.

Por consiguiente, cada VJP local se evalúa exactamente en el punto en que se evaluaría en el grafo completo, y su composición —que es precisamente la regla de la cadena— produce el mismo valor. Finalmente, LIBERAR(W_i) solo reubica un tensor; la aritmética es invariante a la ubicación en el dispositivo. ■

Corolario 4 (No es una aproximación). *USF no introduce error, sesgo ni varianza. Cualquier desviación respecto del gradiente del grafo completo se limita al no determinismo ordinario de la aritmética en punto flotante (Observación 6).*

Corolario 5 (Agnosticismo al adaptador). *El Teorema 3 no emplea ninguna propiedad de f_i más allá de su diferenciabilidad. USF es por tanto independiente del adaptador: se aplica literalmente a LoRA, a adaptadores espectrales o a cualquier parametrización de θ . Lo verificamos con dos adaptadores distintos en la Sección XI.*

Observación 6 (Consideraciones sobre punto flotante). *El Teorema 3 afirma la igualdad en aritmética exacta; dos puntos merecen atención en la práctica. Primero, los pesos se preservan exactamente: CARGAR relee el mismo tensor fp16, de modo que no se produce recuantización, y esto se cumple incondicionalmente. Segundo, si la recomputación selecciona un kernel distinto del pase hacia adelante original, el orden de reducción en punto flotante puede diferir en los últimos bits. Este es el no determinismo inherente al checkpointing de activaciones estándar [13] y no es específico de USF. En nuestras mediciones, con formas y kernels idénticos, la concordancia es exacta (Tabla III).*

V. LA COTA DE MEMORIA

Teorema 7 (Memoria pico). *Bajo el Algoritmo 1 con granularidad $\mathcal{U}$, la memoria pico del dispositivo satisface*

$$\text{VRAM}_{\text{pico}} \leq \underbrace{\max_u |W_u| \cdot s}_{\text{unidad residente}} + \underbrace{|\theta| \cdot s_{\text{opt}}}_{\text{adaptadores}} + \underbrace{A}_{\text{activaciones}} + \underbrace{K}_{\text{checkpoints}}, \quad (4)$$

donde A es la memoria de activaciones internas de una única unidad y $K = L \cdot B \cdot S \cdot d \cdot s$ si los checkpoints $\{c_i\}$ residen en el dispositivo.

Demostración: En todo instante del Algoritmo 1 hay exactamente una unidad cargada, dado que CARGAR y LIBERAR están estrictamente anidados y no se solapan (Definición 2); de ahí el primer término. Los adaptadores son residentes por construcción: θ se actualiza en cada paso y su estado del optimizador debe persistir entre pasos; su tamaño es $|\theta| \ll |W|$ por la hipótesis PEFT. Un grafo con gradiente existe únicamente dentro de la iteración de una unidad, porque el pase hacia adelante no construye grafo; de ahí A . Por último, los c_i son los únicos tensores que sobreviven entre iteraciones. ■

Corolario 8 (Independencia de la profundidad). *Los checkpoints c_i son activaciones, y a diferencia de los pesos no se requieren en el dispositivo entre su producción y su uso. Descargarlos a la memoria del anfitrión da $K \approx 0$ y por tanto*

$$\text{VRAM}_{\text{pico}} \approx \max_u |W_u| \cdot s + |\theta| \cdot s_{\text{opt}} + A, \quad (5)$$

que es independiente de L . Añadir capas a un modelo no incrementa su requisito de memoria.

Corolario 9 (Independencia del tamaño del modelo). *La cota no contiene $\sum_i |W_i|$. Dos modelos cuya unidad más grande tenga el mismo tamaño presentan el mismo requisito de memoria, por muy distintos que sean sus números totales de parámetros.*

El Corolario 9 es la afirmación central del artículo. No dice que un modelo mayor sea tan barato de ajustar como uno

menor: los costes de E/S y de cómputo escalan ambos con $\sum_i |W_i|$, como cuantifica la Sección IX. Dice que esos costes se pagan en tiempo, no en memoria. El requisito del dispositivo lo fija la unidad individual más grande y nada más, razón por la cual la jerarquía de granularidad de la sección siguiente determina la viabilidad.

VI. GRANULARIDAD

Teorema 10 (Jerarquía de granularidad). *Para las particiones anidadas de la Definición 1,*

$$\max_{i,p} |W_{i,p}| \leq \max_i \max \left(|W_i^{\text{atm}}|, |W_i^{\text{mlp}}| \right) \leq \max_i |W_i|, \quad (6)$$

y el Teorema 7 se aplica a cualquiera de ellas. Refinar la granularidad reduce la memoria pico de forma monótona con idéntica E/S total.

Demostración: Las particiones están anidadas (matriz $\subset$ sub-capa $\subset$ capa), y un máximo sobre una partición más fina no puede exceder el máximo sobre una más gruesa. La E/S total por pase es $\sum_u |W_u| = \sum_i |W_i|$, invariante a la partición: los mismos bytes, en fragmentos menores. ■

La Tabla I instancia el Teorema 10 para cuatro familias de modelos. Se siguen dos observaciones. Llama-3-70B [19] cabe en una tarjeta de 3,94 GB con granularidad de *capa*, con 1,59 GB por bloque; no se requiere refinamiento alguno. Un modelo de 405 B no cabe con granularidad de *capa* ni de sub-capa, pero su matriz individual más grande ocupa 1,62 GB y sí cabría, que es el contenido del Corolario 9 llevado a su extremo.

Observación 11 (Límite de implementación). *El Teorema 10 es un enunciado sobre particiones y se cumple con granularidad de matriz. Nuestra implementación, sin embargo, admite únicamente granularidad de capa y de sub-capa. La razón es de ingeniería y no matemática: delegamos la atención en la implementación de referencia [20], que consume W_q, W_k, W_v, W_o conjuntamente en una única llamada. Transmitirlos de uno en uno exigiría reimplementar los mecanismos internos de la atención, incluidos los embeddings rotatorios [21] y la atención de consulta agrupada [22], sustitución que juzgamos portadora de más riesgo de incorrección silenciosa que el beneficio que reportaría. Afirmamos, por tanto, 70B por la cota con una granularidad implementada, y no afirmamos nada sobre 405B más allá de la Tabla I.*

VII. STREAMING LAYER-MAJOR

El Algoritmo 1 es *batch-major*: por cada micro-lote recorre las L unidades. Con acumulación sobre G micro-lotes, el modelo se lee por tanto G veces por paso de optimizador. El siguiente reordenamiento elimina ese factor.

Definición 12 (Layer-major). *Intercámbiense los dos bucles: manténgase la unidad u residente y pásense todos los G micro-lotes por ella antes de avanzar a $u+1$. El pase hacia atrás es simétrico: una carga de W_i y después G retropropagaciones locales.*

Tabla I

TAMAÑO DE UNIDAD POR GRANULARIDAD (FP16). LOS ADAPTADORES, ACTIVACIONES Y CHECKPOINTS AÑADEN MENOS DE 0,6 GB. LOS VALORES SE COMPUTAN A PARTIR DE LAS CONFIGURACIONES DE ARQUITECTURA PUBLICADAS MEDIANTE EL MISMO CÓDIGO QUE GOBIERNA EL STREAMING.

Modelo	L	capa	attn/mlp	matriz	¿4 GB?
Qwen2.5-7B	28	0,43 GB	0,05 / 0,38	0,13 GB	cualquiera
Llama-3-8B	32	0,41 GB	0,08 / 0,33	0,11 GB	cualquiera
Clase 13B	40	0,59 GB	0,20 / 0,40	0,13 GB	cualquiera
Llama-3-70B	80	1,59 GB	0,28 / 1,31	0,44 GB	capa
Llama-3-405B	126	5,94 GB	1,06 / 4,88	1,62 GB	solo matriz†

† La granularidad de matriz no está implementada; véase la Observación 11.

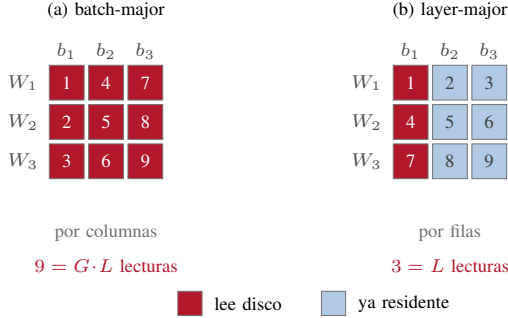


Figura 1. Por qué invertir el orden de los bucles divide la E/S por G (Teorema 13), dibujado para $L=3$ unidades y $G=3$ micro-lotes. Las celdas son visitas (unidad, micro-lote), numeradas en orden de recorrido; ambas planificaciones realizan las nueve, de modo que ambas computan el mismo gradiente. Batch-major (a) termina un micro-lote antes de avanzar, así que cada visita relea su unidad desde disco. Layer-major (b) termina una unidad antes de avanzar, así que solo la primera visita de cada fila lee: las otras $G-1$ encuentran los pesos ya ahí. El conteo cae de $G \cdot L$ a L , y el precio son G conjuntos de activaciones de frontera, que el Corolario 8 ubica en la memoria del anfitrión.

La Figura 1 muestra por qué esto importa. Ambas planificaciones visitan exactamente el mismo conjunto de pares (unidad, micro-lote) y, por el Teorema 3, cada visita computa el mismo VJP local; difieren únicamente en el orden del recorrido y, por tanto, en cuántas de esas visitas encuentran la unidad ya residente.

Teorema 13 (Exactitud y E/S de layer-major). *La planificación layer-major de la Definición 12 produce gradientes idénticos a los del Algoritmo 1, y su E/S por paso de optimizador es*

$$E/S_{\text{layer-major}} = 2 \sum_i |W_i|, \quad (7)$$

$$E/S_{\text{batch-major}} = G \cdot 2 \sum_i |W_i|, \quad (8)$$

una reducción por el factor G .

Demostración: Exactitud. La acumulación de gradiente computa $\nabla_{\theta} \ell = \sum_{j=1}^G \nabla_{\theta} \ell^{(j)}$. La suma es conmutativa y asociativa, de modo que el orden en que se acumulan los términos no afecta al resultado. Cada $\nabla_{\theta} \ell^{(j)}$ se computa mediante el VJP local del Teorema 3, evaluado en $(c_i^{(j)}, W_i, \theta_i)$: los mismos puntos que en la planificación batch-major. Intercambiar los bucles solo reordena estas visitas; deja intacto cada punto de evaluación.

E/S. En la Definición 12 el bucle externo recorre i una sola vez por paso de optimizador, de modo que cada W_i se carga exactamente una vez por pase y dos en total (adelante y atrás), lo que da $2 \sum_i |W_i|$. En el Algoritmo 1 el bucle externo sobre j repite el recorrido completo G veces. ■

El coste es que deben mantenerse G conjuntos de activaciones en cada frontera de unidad, $G \cdot B \cdot S \cdot d \cdot s$ bytes; para $G = 8$, $B = 4$, $S = 256$, $d = 3584$ en fp16 esto son 59 MB por frontera, y por el Corolario 8 pueden ubicarse en la memoria del anfitrión. El intercambio es, por tanto, G veces menos E/S a cambio de G veces más activaciones descargables al anfitrión.

Bajo layer-major, el tiempo por ejemplo es

$$T_{\text{ej}}(G) = \frac{2 \sum_i |W_i|}{\text{BW} \cdot G \cdot B} + \frac{C}{B}, \quad (9)$$

con C el tiempo de cómputo de un micro-lote y BW el ancho de banda efectivo de carga de pesos, medido de extremo a extremo a través de CARGAR, no la velocidad cruda de lectura del disco (la Sección IX cuantifica la brecha). G solo aparece en el término de E/S, en el denominador, así que T_{ej} decrece monótonamente en G hacia el límite de cómputo C/B : el mayor G que el presupuesto de memoria de activaciones permita es óptimo, sin que haga falta ningún argumento adicional.

Observación 14 (Por qué está ausente del trabajo previo). *Layer-major carece de sentido cuando el modelo es residente: si W_i nunca abandona el dispositivo, cargarlo «una vez» en lugar de « G veces» no es una distinción. El reordenamiento se convierte en la optimización dominante precisa y únicamente en el régimen de streaming, que es el régimen que este artículo construye.*

VIII. PREFETCH

Proposición 15 (Condición de solapamiento). *Sean $t_{io}(u) = |W_u|/\text{BW}$ y $t_{\text{cmp}}(u)$ el tiempo de cómputo de la unidad u . Si $\text{CARGAR}(u+1)$ se emite de forma asíncrona al comenzar el cómputo de u , entonces*

$$T_{\text{paso}} \approx \max(\sum_u t_{io}(u), \sum_u t_{\text{cmp}}(u)) + t_{io}(u_1), \quad (10)$$

y la E/S queda totalmente oculta siempre que $t_{io}(u+1) \leq t_{\text{cmp}}(u)$ para todo u .

Demostración: La planificación es un pipeline de dos etapas (carga; cómputo) con búferes independientes. En régimen estacionario su rendimiento es el de la etapa más lenta, y el llenado inicial aporta $t_{io}(u_1)$. ■

El doble búfer mantiene dos unidades residentes, de modo que el primer término de (4) pasa a ser $2 \max_u |W_u| \cdot s$; para 7 B con granularidad de capa esto son 0,86 GB en lugar de 0,43 GB, lo que sigue siendo holgado en una tarjeta de 3,94 GB. Con nuestras constantes calibradas (Sección IX), un recorrido completo cuesta $\sum_u t_{io} \approx 48$ s frente a $\sum_u t_{\text{cmp}} \approx 57$ s para un único micro-lote con $B=4$, $S=256$, de modo que la condición de solapamiento se cumple —aunque por poco, lo que concuerda con (13), que sitúa el umbral de cómputo en $8,6 \times 10^2$ tokens mientras esta configuración aporta 1024. Combinado con el Teorema 13, que divide t_{io} por G , el margen

pasa a ser amplio. El prefetch altera cuándo se lee un tensor, nunca su valor, por lo que el Teorema 3 no se ve afectado; hecho que verificamos en lugar de suponer (Tabla III).

IX. MODELO DE COSTE

Sean $N = \sum_i |W_i|/s$ el número de parámetros transmitidos, Φ el rendimiento efectivo del dispositivo, y BW el ancho de banda efectivo de *carga de pesos*: el throughput medido de pared de CARGAR de extremo a extremo —lectura del archivo, deserialización de safetensors y copia al dispositivo— no la velocidad de lectura secuencial cruda del disco. La distinción importa: una lectura cruda de un fragmento del modelo en nuestro NVMe alcanza 1,3–1,7 GB/s (dd, O_DIRECT), aproximadamente $3 \times$ nuestro BW = 0,54 GB/s calibrado; la brecha es sobrecarga del lado de Python y de la deserialización dentro de CARGAR, no una propiedad del disco. La (11) usa los 0,54 GB/s calibrados en todo momento, pues eso es lo que realmente limita un paso de entrenamiento, no la capacidad cruda del disco. Empleando la estimación estándar de $6N$ FLOPs por token para un pase combinado hacia adelante y hacia atrás [23],

$$T_{io} = 2Ns/BW \quad (\text{layer-major}), \quad (11)$$

$$T_{cmp} = 6N(G \cdot B \cdot S)/\Phi, \quad (12)$$

con $T_{paso} = T_{io} + T_{cmp}$ sin prefetch y $\max(T_{io}, T_{cmp})$ con él (Proposición 15). El sistema está limitado por cómputo cuando $T_{cmp} > T_{io}$; sustituyendo (11)–(12), el tamaño transmitido N se cancela y la condición se reduce a un enunciado sobre tokens únicamente,

$$G \cdot B \cdot S > \frac{s \Phi}{3BW}. \quad (13)$$

Que N se cancele es lo sustantivo: qué recurso limita es una propiedad de la *planificación* y no del tamaño del modelo, de modo que un modelo mayor no queda más limitado por E/S, sino uniformemente más lento. Con nuestras constantes calibradas ($s = 2$, $\Phi = 0,7$ TFLOP/s, $BW = 0,54$ GB/s) el umbral es $\approx 8,6 \times 10^2$ tokens por paso. Un paso layer-major con $G = 8$, $B = 4$, $S = 256$ procesa 8192 tokens y está por tanto firmemente limitado por cómputo, con un orden de magnitud de margen: el reordenamiento del Teorema 13 no se limita a acelerar el sistema, cambia qué recurso lo limita. Este es el razonamiento roofline de [16] aplicado a un bucle de entrenamiento *out-of-core*.

Validación. Calibramos BW y Φ a partir de dos mediciones independientes ($B = 1$ y $B = 4$) y empleamos el modelo para predecir el resto. La Tabla II recoge el resultado: la mayor discrepancia es del 6 %. Nos apoyamos en esta concordancia en la Sección XII, donde el modelo se usa de forma predictiva.

X. SELECCIÓN AUTOMÁTICA DE GRANULARIDAD

Los Teoremas 7 and 10 proporcionan un criterio operativo. Dado un presupuesto de memoria V , recórranse las granularidades de la más gruesa a la más fina y devuélvase la primera cuya cota quepa.

Tabla II
MODELO DE COSTE FRENTE A MEDICIÓN (QWEN2.5-7B, GTX 1050, $B = 4$, $S = 256$).

Magnitud	Modelo	Medido	Error
E/S por paso (batch-major)	24,3 GB	24,9 GB	2 %
Tiempo por ejemplo (batch-major)	25,6 s	26,9 s	5 %
Aceleración layer-major ($G=4$)	1,49×	1,40×	6 %

Proposición 16 (Optimalidad de la granularidad más gruesa). *La granularidad más gruesa que satisface (4) minimiza el tiempo sujeto a la restricción de memoria.*

Demostración: La E/S total es invariante a la granularidad (Teorema 10), pero el número de operaciones CARGAR crece conforme se refina la partición. Cada una acarrea una sobrecarga fija λ (acceso a fichero, lanzamiento), de modo que el tiempo es $|\mathcal{U}|\lambda + 2Ns/BW$: el segundo término es constante y el primero crece con $|\mathcal{U}|$. Por consiguiente, la partición factible más gruesa es óptima. ■

La consecuencia es que no se requiere configuración por parte del usuario. El sistema lee la descripción de la arquitectura, computa $\max_u |W_u|$ para cada granularidad candidata y selecciona una; la misma ruta de código sirve a un modelo de 7 B y a uno de 70 B en la misma tarjeta, difiriendo únicamente en la granularidad elegida. Cuando ninguna granularidad admitida cabe, el sistema lo comunica en lugar de fallar de forma opaca y, conforme a la Observación 11, indica explícitamente cuándo una granularidad no implementada habría bastado.

XI. EXPERIMENTOS

XI-A. Configuración

Todos los experimentos se ejecutan en un único equipo portátil con una NVIDIA GeForce GTX 1050 móvil (GP107M, Pascal, 3,94 GB utilizables, sin tensor cores), 8 núcleos de CPU, 15,5 GB de memoria de anfitrión y un disco NVMe. Elegimos esta tarjeta deliberadamente: no es una opción razonable para entrenar a gran escala, y ese es precisamente el punto—si la cota del Teorema 7 se cumple aquí, se cumple en cualquier GPU que un practicante sin acceso a un centro de datos pueda razonablemente poseer. El modelo es Qwen2.5-7B-Instruct [24] (7,62 B parámetros, 28 capas, fp16), ajustado sobre Alpaca [25] con AdamW [26], $S = 256$, precisión mixta [27], en PyTorch [28] con Transformers [20]. Se emplean dos adaptadores para sustanciar el Corolario 5: LoRA [1] con rangos 1, 2 y 8, y S^3 , un adaptador *spectral-spatial-smooth* (espectral, espacial y suave, de donde las tres eses) descrito en un artículo complementario. Este trabajo no formula afirmación alguna sobre la calidad relativa de ambos; aparecen aquí como evidencia de que USF es indiferente a esa elección.

La implementación, las configuraciones de los experimentos y los registros en bruto que respaldan cada número de este artículo están disponibles en una revisión etiquetada e inmutable [29]: el desarrollo del método continúa en otro punto de ese repositorio, pero la etiqueta fija el código exacto que produjo las mediciones aquí reportadas. Cada afirmación de exactitud de la Tabla III se corresponde con un test ejecutable, y las

Tabla III

DESVIACIÓN DE GRADIENTE MEDIDA PARA CADA AFIRMACIÓN DE EXACTITUD. «TENSORES» ES EL NÚMERO DE TENSORES DE GRADIENTE ENTRENABLES COMPARADOS.

Afirmación	Comparación	Tensores	máx $ \Delta $
Teorema 3	transmitido vs. residente	150	0,00
Teorema 13	layer- vs. batch-major (7B)	1400	0,00
Corolario 8	checkpoints en anfitrión vs. GPU	150	0,00
Teorema 10	sub-capa vs. capa	150	0,00
Proposición 15	con vs. sin prefetch	150	0,00

figuras se regeneran desde la ejecución registrada en lugar de redibujarse, de modo que las desviaciones reportadas pueden reproducirse y no exigen confianza.

XI-B. Exactitud

El Teorema 3, el Teorema 13, el Teorema 10 y la Proposición 15 afirman, cada uno, que una transformación deja los gradientes inalterados. La Tabla III recoge la desviación medida para cada una. Todas las entradas son exactamente cero.

La segunda fila merece énfasis. El Teorema 3 solo puede verificarse frente a una línea base residente en un modelo que *quepa* residente, lo que en este hardware significa uno pequeño; lo verificamos por tanto en una configuración reducida. El Teorema 13, en cambio, compara dos planificaciones de streaming y es verificable a escala completa: sobre el propio Qwen2.5-7B, en 1400 tensores de gradiente cuya magnitud máxima es $7,55 \times 10^2$, la desviación es exactamente cero. La escala de la comparación importa porque excluye la posibilidad de que la concordancia sea un artefacto de un modelo pequeño.

XI-C. Memoria

La Tabla IV recoge la memoria pico del dispositivo. Ambos adaptadores entrenan el mismo modelo congelado de 7 B en la misma tarjeta. La diferencia entre ellos es precisamente el término $|\theta| \cdot s_{\text{opt}} + A$ de (4) — S^3 mantiene factores residentes adicionales— y ambos satisfacen la cota. Que LoRA ocupe *menos* memoria que S^3 no es un defecto del sistema sino una confirmación del Corolario 5: USF impone su cota con independencia de qué se esté adaptando.

Reportamos tres medidas en lugar de una porque difieren de forma sustancial y solo la mayor constituye una prueba justa de la afirmación. La cifra destacada de 2,2 GB es el pico del asignador; la ocupación total del dispositivo incluyendo el contexto CUDA alcanza 2891 MB, esto es, el 71,7 % de los 4034 MB que la tarjeta expone a CUDA. La afirmación sobrevive bajo la medida más estricta, que es precisamente la razón para enunciarla. Nótese además que la memoria pico no es monótona en el tamaño del adaptador —LoRA $r=1$ ocupa más que $r=2$ bajo *resv* y *smi*— porque a esta escala la fragmentación del asignador pesa mucho más que el número de parámetros. El Teorema 7 acota la asignación misma; la política de caché del asignador es una cuestión aparte.

Tabla IV

MEMORIA PICO Y PARÁMETROS ENTRENABLES (QWEN2.5-7B, 15,2 GB EN DISCO). SE REPORTAN TRES MEDIDAS: *asig* ES EL PICO DEL ASIGNADOR DE PYTORCH, *resv* CONTABILIZA ADEMÁS LOS BLOQUES QUE EL ASIGNADOR RETIENE EN CACHÉ, Y *smi* ES LA OCUPACIÓN TOTAL DEL DISPOSITIVO INCLUYENDO EL CONTEXTO CUDA —LA MEDIDA MÁS ESTRICTA, Y LA QUE DEBE CABER. LA PERPLEJIDAD SE MIDE SOBRE 100 EJEMPLOS RESERVADOS DE ALPACA, CON DATOS Y SEMILLA IDÉNTICOS EN TODAS LAS FILAS.

Adaptador	Entren.	VRAM pico (MB)			Perplejidad	
		asig	resv	smi	antes	después
S^3	3,90 M	2269	2514	2891	9,52	4,46
LoRA $r=8$	20,19 M	1606	2072	2554	9,52	4,54
LoRA $r=2$	5,05 M	1598	1780	2161	9,52	4,57
LoRA $r=1$	2,52 M	1315	1938	2351	9,52	4,58
Capacidad de la tarjeta		4034				

Tabla V

BATCH-MAJOR FRENTE A LAYER-MAJOR (QWEN2.5-7B, $B=4$, $S=256$, $G=4$, 16 EJEMPLOS).

Planificación	Total	Por ejemplo	VRAM pico
Batch-major	457,5 s	28,6 s	2524 MB
Layer-major	327,4 s	20,5 s	2219 MB
Ganancia		1,40×	−305 MB

XI-D. Aprendizaje de extremo a extremo

La Tabla IV establece que el entrenamiento reduce la perplejidad en datos reservados para todos los adaptadores. Una ejecución más larga —1100 ejemplos de Alpaca, una época, 275 micro-lotes con $B=4$, esto es 138 pasos de optimizador con $G=2$, 4 h 16 min— redujo la perplejidad reservada de 11,96 a 5,50 con un pico de 2233 MB, sin ningún paso omitido por desbordamiento de gradiente. La Figura 3 traza la trayectoria. Presentamos estos datos como evidencia de que el sistema entrena, no como una afirmación de calidad: por el Teorema 3 el modelo resultante es el modelo que el entrenamiento ordinario habría producido, de modo que no hay nada relativo a la calidad que USF pudiera alterar.

La Figura 2 muestra la traza de memoria del dispositivo de esa ejecución, y es la imagen empírica más directa del Teorema 7. El diente de sierra es el mecanismo mismo: cada diente es un ciclo CARGAR—computar—LIBERAR, y la envolvente nunca se aproxima a los 4034 MB de capacidad de la tarjeta pese a que el modelo ocupa 15,2 GB en disco. La traza expone también el comportamiento del asignador: la curva reservada deriva al alza conforme el asignador en caché retiene bloques liberados, razón por la cual reportamos las tres medidas en la Tabla IV y no la más favorable.

XI-E. Layer-major

La Tabla V compara ambas planificaciones sobre el modelo de 7 B en 16 ejemplos con $G = 4$. Layer-major es $1,40\times$ más rápido y emplea *menos* memoria, esto último porque sus checkpoints residen en el anfitrión (Corolario 8). La (9) predice mayores ganancias con G superior; no las medimos.

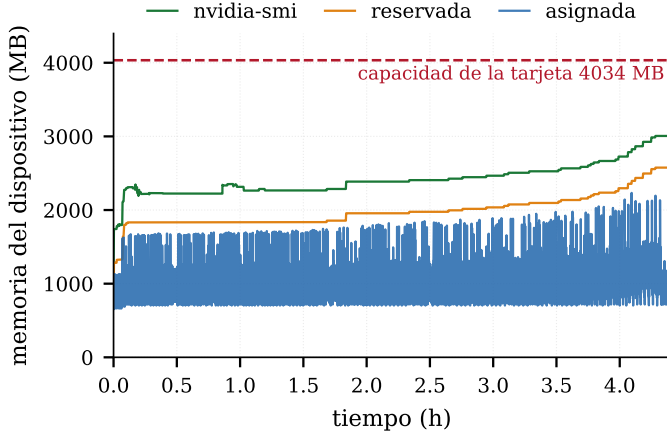


Figura 2. Memoria del dispositivo a lo largo de la ejecución completa de 4 h 16 min de Qwen2.5-7B (114 766 muestras cada 100 ms). El diente de sierra es el Algoritmo 1 mismo: cada diente es un ciclo CARGAR–computar–LIBERAR de una capa decodificadora de 0,43 GB. El modelo ocupa 15,2 GB en disco y, aun así, nada se aproxima a los 4034 MB de la tarjeta —la imagen que el Teorema 7 predice. La deriva al alza de la curva reservada es el asignador en caché reteniendo bloques liberados, no un crecimiento de lo que USF mantiene residente.

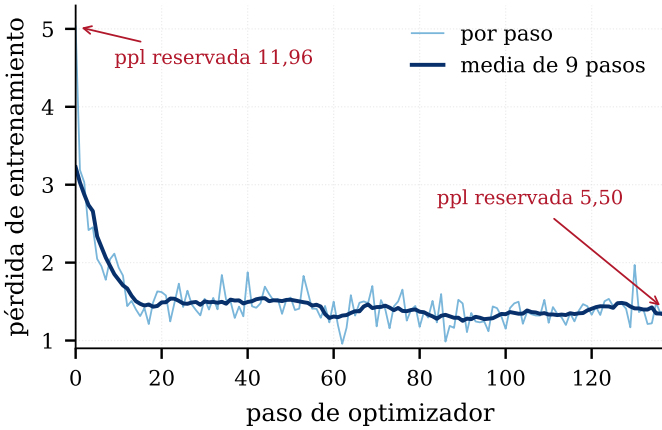


Figura 3. Pérdida de entrenamiento a lo largo de 138 pasos de optimizador (275 micro-lotes con $B=4$, $G=2$) sobre 1100 ejemplos de Alpaca. La perplexidad en datos reservados, medida sobre 100 ejemplos no vistos antes y después, cayó de 11,96 a 5,50. Ningún paso fue omitido por desbordamiento de gradiente. Por el Teorema 3 esta trayectoria es la que habría producido el entrenamiento ordinario con residencia: lo que la curva demuestra es que el sistema aprende; qué tan bien lo hace queda fuera del alcance de este artículo.

XI-F. Comparación con la cuantización

La alternativa natural para reducir la memoria residente es la cuantización. En este presupuesto no es meramente inferior: es inviable. Una cuantización NF4 a 4 bits de Qwen2.5-7B requiere 3,25 GB para los bloques transformer, más 1,09 GB para la matriz de embeddings y otros 1,09 GB para la cabeza de modelado de lenguaje, que este modelo no ata al embedding y que la implementación de referencia no cuantiza: 5,43 GB frente a una tarjeta que expone 3,94 GB. La predicción se confirma —la carga termina en un error de memoria agotada— mientras que USF entrena el mismo modelo con un pico de 2,27 GB en el asignador.

La comparación está trazada deliberadamente a favor de la cuantización. Los 5,43 GB contabilizan únicamente pesos,

Tabla VI
ALCANCE DE LAS AFIRMACIONES. LAS TRES FILAS POSEEN DISTINTO ESTATUS EPISTÉMICO.

Modelo	Estatus	Base
Qwen2.5-7B	Demostrado	Ejecutado: 2,2 GB de pico, ppl $9,52 \rightarrow 4,46$, máx $ \Delta = 0$
Llama-3-70B	Predicho por la cota; no ejecutado	1,59 GB por capa $< 3,94$ GB con una granularidad <i>implementada</i> (Tabla I); no dispusimos de los pesos
Llama-3-405B	Teórico; requiere una extensión no implementada	Precisa granularidad de matriz (1,62 GB); véase la Observación 11

antes del estado del optimizador, las activaciones o el contexto CUDA, y son por tanto una cota inferior de lo que 4 bits necesitaría; las cifras de USF son tiempo de ejecución medido y completo. La distancia es lo bastante amplia como para que la asimetría no altere nada, y por eso podemos enunciar la conclusión como una separación de viabilidad y no como una preferencia. Ambas técnicas son comonibles y abordan ejes distintos: la cuantización compra memoria con precisión, USF la compra con tiempo, y solo esta última es exacta (Corolario 4).

XII. ANÁLISIS DE ESCALADO

La Tabla VI enuncia qué establece este artículo en cada escala y sobre qué base.

Empleando (11)–(12) con las constantes calibradas en la Sección IX, y layer-major con $G = 8$, el coste proyectado de 1000 ejemplos es de 4,0 h para 7 B, 41,7 h para 70 B y 245 h para 405 B; los tres están limitados por cómputo, de modo que las proyecciones las gobierna Φ y no el ancho de banda de disco. La lectura honesta de estas cifras es que USF elimina la barrera de memoria y la sustituye por una barrera de tiempo. Un modelo de 70 B pasa a ser viable como demostración en una tarjeta de consumo; no pasa a ser un flujo de trabajo práctico. El régimen práctico para esta clase de hardware es de 7 B a 13 B, y así lo hacemos constar en la Sección XIII en lugar de dejarlo a que el lector lo descubra.

XIII. LIMITACIONES

El tiempo sustituye a la memoria. USF es sustancialmente más lento que el entrenamiento residente; el método entero es un intercambio, y la Sección XII lo cuantifica. Afirmamos viabilidad en hardware donde la alternativa es no hacer nada en absoluto, no competitividad con una GPU de centro de datos.

Se requiere almacenamiento local. El método presupone que los pesos están en un disco local, y BW acota el término de E/S de (11).

La hipótesis PEFT es estructural. El Teorema 7 trata $|\theta| \cdot s_{\text{opt}}$ como pequeño. Con muchos parámetros entrenables, el estado del optimizador deja de ser despreciable y su particionado pasa a ser la preocupación dominante, precisamente el régimen que aborda ZeRO [10]. USF apunta a PEFT; el preentrenamiento queda fuera de su alcance.

La granularidad de matriz no está implementada (Observación 11), razón por la cual la Tabla VI asigna a 405 B un estatus más débil que a 70 B.

Las constantes son específicas del hardware. El *modelo* de coste de la Sección IX es general; los valores de Φ y BW empleados en las Tablas II and VI no lo son, y requerirían recalibración en otro equipo.

La inferencia no se ve afectada. USF es un método de entrenamiento.

XIV. CONCLUSIÓN

Hemos mostrado que la memoria necesaria para ajustar un modelo de lenguaje congelado no es función del tamaño del modelo. Es función de su unidad transmisible más grande, porque la diferenciación en modo inverso es local: nada en la corrección ata un peso al dispositivo una vez computado su VJP, así que cuánto tiempo permanece residente es algo que nos toca a nosotros planificar. El método resultante es exacto —verificado con desviación nula sobre el propio modelo de 7 B—, agnóstico al adaptador y gobernado por un modelo de coste preciso dentro de un 6 %. Su consecuencia es un desplazamiento, y no una eliminación, del coste: el tamaño del modelo pasa a ser una cuestión de tiempo y almacenamiento. En una GPU de consumo que un centro de datos consideraría obsoleta, un modelo de 7 B puede ahora ajustarse de forma exacta; la misma cota, con una granularidad implementada, sitúa un modelo de 70 B al alcance de esa misma tarjeta. La barrera que queda es el tiempo, que la Sección XII cuantifica y que —a diferencia de la memoria— no excluye a nadie de empezar.

REFERENCIAS

- [1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [2] N. Houshy, A. Giurugi, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for NLP,” in *International Conference on Machine Learning (ICML)*, 2019.
- [3] B. Lester, R. Al-Rfou, and N. Constant, “The power of scale for parameter-efficient prompt tuning,” in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.
- [4] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [5] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [6] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, “ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.
- [7] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. SIAM, 2008.
- [8] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic differentiation in machine learning: a survey,” *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018.
- [9] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen, “DoRA: Weight-decomposed low-rank adaptation,” in *International Conference on Machine Learning (ICML)*, 2024.
- [10] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO: Memory optimizations toward training trillion parameter models,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.
- [11] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. Zhang, D. Li, and Y. He, “ZeRO-Offload: Democratizing billion-scale model training,” in *USENIX Annual Technical Conference (ATC)*, 2021.
- [12] S. Gugger, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan, “Accelerate: Training and inference at scale made simple, efficient and adaptable,” <https://github.com/huggingface/accelerate>, 2022.
- [13] T. Chen, B. Xu, C. Zhang, and C. Guestrin, “Training deep nets with sublinear memory cost,” *arXiv preprint arXiv:1604.06174*, 2016.
- [14] A. Griewank and A. Walther, “Algorithm 799: Revolve: An implementation of checkpointing for the reverse or adjoint mode of computational differentiation,” *ACM Transactions on Mathematical Software*, vol. 26, no. 1, pp. 19–45, 2000.
- [15] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [16] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [17] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, “GPipe: Efficient training of giant neural networks using pipeline parallelism,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [18] Z. Jia, M. Zaharia, and A. Aiken, “Beyond data and model parallelism for deep neural networks,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2019.
- [19] A. Grattafiori, A. Dubey, A. Jauhri *et al.*, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [20] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue *et al.*, “Transformers: State-of-the-art natural language processing,” in *Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020.
- [21] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, 2024.
- [22] N. Shazeer, “Fast transformer decoding: One write-head is all you need,” *arXiv preprint arXiv:1911.02150*, 2019.
- [23] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [24] Qwen Team, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2025.
- [25] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, “Stanford Alpaca: An instruction-following LLaMA model,” https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [26] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [27] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia *et al.*, “Mixed precision training,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [28] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [29] “USF: Reference implementation of unit-streamed fine-tuning,” <https://github.com/Anndy-bit/lowrank-field-adapters/tree/v1.0-usf-paper1>, 2026, source code, experiment configurations and run logs. Tagged release v1.0-usf-paper1 (commit 9a86aed): the exact revision that produced every measurement reported here.